Électronique & Interfaçage

Initiation à l'Arduino

4ème année Génie Informatique

19 octobre 2026

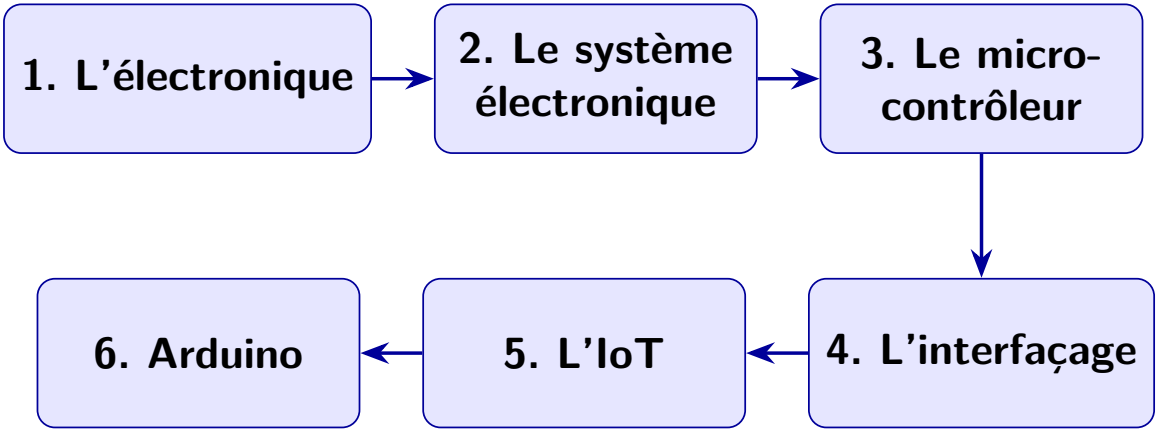
Ouverture

- Positionnement de l'UE dans le cursus (4ème année Génie Informatique)
- Objectifs pédagogiques
- Annonce du projet pratique (fil rouge)
- Plan des 4 parties

Plan

1. Partie 1 - Concepts : l'électronique et l'interfaçage
2. Partie 2 - Prise en main d'un kit Arduino
3. Partie 3 - Programmation Arduino
4. Partie 4 - Étude de cas

Organisation :

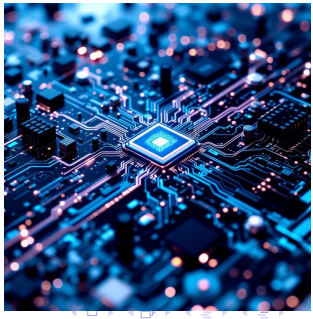


C'est quoi l'électronique ?

Définition

L'**électronique** est la discipline technique qui consiste à **contrôler la circulation des électrons** dans un conducteur afin de générer, traiter ou transmettre un courant électrique pour réaliser une fonction précise.

- **Une science du signal, pas de la physique**
  - **Approche systémique** : on raisonne en fonctions (amplifier, filtrer, convertir).
  - **Boîtes noires** : seules les entrées (signaux) et sorties (résultats) comptent.
  - **Électronique** $\neq$ **Électricité** : l'une traite l'**information**, l'autre transporte l'**énergie**.
- **Présente partout** : téléphone, télévision, voiture, machine à laver, carte bancaire...



Les composants de base : les « briques » de l'électronique

Composant	**Son rôle (en une phrase)**
Résistance	Limite le courant, comme un robinet à moitié fermé
LED	Diode qui s'allume quand le courant la traverse
Condensateur	Petit réservoir qui stocke et restitue de l'énergie
Transistor	Interrupteur commandé électriquement, brique de base de toutes les puces
Capteur	Transforme une grandeur physique (chaleur, lumière...) en signal électrique
Actionneur	Transforme un signal electrique en mouvements
Circuit intégré	Des milliers à des milliards de transistors réunis dans une seule puce

Deux façons de représenter l'information : analogique et numérique

Signal analogique

Varie **de façon continue**.

Exemple : un thermomètre à mercure, une température de 23,4 °C.

Signal numérique

Ne prend que **deux états** : 0 ou 1 (éteint / allumé).

Exemple : un interrupteur, une porte ouverte ou fermée.

Le point clé

Le monde réel est **analogique**, mais les ordinateurs ne comprennent que le **numérique**. Il faut donc un « traducteur » : c'est tout l'enjeu de l'interfaçage.

Étape 2

Des composants...
...au **système**

Assembler les briques pour accomplir une mission.

C'est quoi un système électronique ?

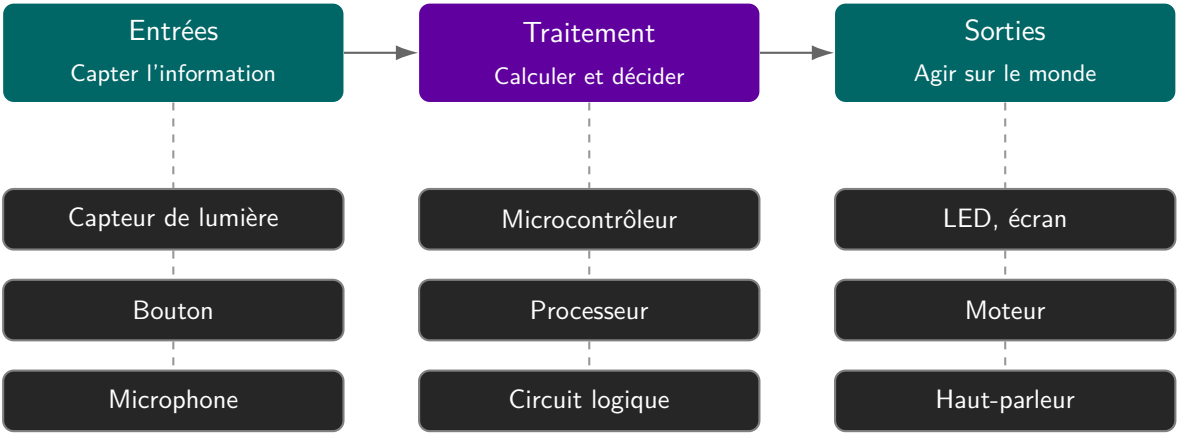
Définition

Un ensemble de composants qui **acquièrent**, **traitent** et **restituent** un signal électrique pour réaliser une fonction précise.

Il s'articule généralement autour de trois blocs :

- **L'entrée :** les capteurs convertissent une grandeur physique en signal électrique.
- **Le traitement :** les microcontrôleurs ou circuits intégrés analysent les données.
- **La sortie :** les actionneurs (écrans, moteurs) traduisent le résultat en action.

Schéma suivi par tous les systèmes électroniques



Capteurs et actionneurs : les « sens » et les « muscles »

Capteurs (les sens)

- Température
- Lumière
- Distance, mouvement
- Humidité du sol
- Bouton, touche

Actionneurs (les muscles)

- LED, écran
- Moteur, ventilateur
- Buzzer, haut-parleur
- Pompe, électrovanne
- Relais (interrupteur)

Il manque encore le « cerveau » qui relie les deux. . .

Étape 3

Le cerveau du système : **le microcontrôleur**

Qui décide ? Qui exécute le programme ?

C'est quoi un microcontrôleur ?

- **Un mini-ordinateur sur une seule puce**
  - Rassemble le processeur, la mémoire et les entrées/sorties (E/S)
- **Fonctions fondamentales**
  - Perçoit via des capteurs, traite par programme et ordonne aux actionneurs
  - Assure la **mesure**, la **commande** et la **régulation**
- **Microcontrôleur vs. Ordinateur (PC)**
  - Dédié à une tâche spécifique
  - Consommation minimale (μW à mW)
  - Peu coûteux, petit, fiable



Figure 1: Microcontroleur STM32.

Et le système embarqué ?

Définition

Un **système embarqué** est un système électronique et informatique **intégré à un objet**, conçu pour exécuter **une tâche précise**, souvent sans que l'utilisateur ne s'en aperçoive.

Vous en utilisez tous les jours :

- Machine à laver, micro-ondes
- Télécommande, montre connectée
- Voiture (ABS, airbags)
- Distributeur de billets

Relation simple

Microcontrôleur = **le cerveau**
Système embarqué = **l'objet complet** qui l'utilise

Étape 4

Faire dialoguer deux mondes : **l'interfaçage**

Le monde physique parle en volts, le programme parle en 0 et 1.

C'est quoi l'interfaçage ?

Définition

L'**interfaçage** est l'ensemble des moyens **mettant en relation** le monde physique (capteurs, actionneurs) et le monde logiciel (programme), afin qu'ils puissent **échanger des informations**.

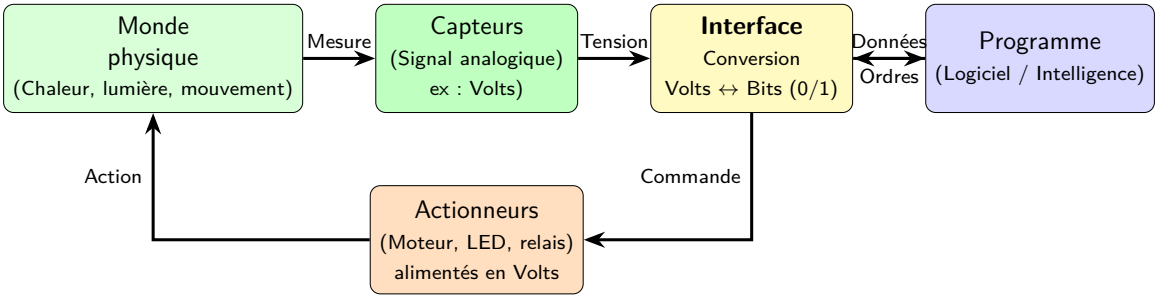
Les missions essentielles :

- **Faire communiquer le monde physique et le logiciel**
  Traduire les phénomènes réels en données exploitables par un programme.
- **Relier capteurs / actionneurs à un programme**
  Convertir les signaux électriques en instructions, et inversement.

Rôle central en Génie Informatique

Indispensable pour l'embarqué, l'IoT, la robotique et les systèmes temps réel.

L'interfaçage en image : un interprète entre deux langues



Comment un programme « lit » une température ?

1. Le capteur de température produit une **tension** (par exemple 0 à 5 V).
2. Le **convertisseur analogique-numérique (CAN)** transforme cette tension en un **nombre** (par exemple de 0 à 1023).
3. Le programme compare ce nombre à un seuil et **décide**.
4. Il envoie un **ordre** (0 ou 1) vers l'actionneur.

En une phrase

Le CAN est le **traducteur** qui transforme une valeur continue du monde réel en un nombre que le microcontrôleur comprend.

Étape 5

Quand les objets **se connectent**

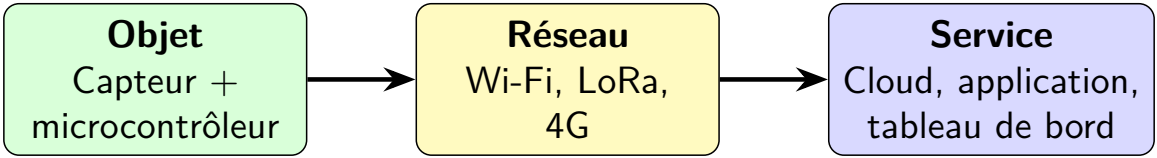
Et si nos systèmes pouvaient aussi parler entre eux, via Internet ?

C'est quoi l'IoT ?

- **IoT = Internet of Things (Internet des objets)**
  - Des objets intelligents et connectés : capter, transmettre, exploiter
- **Architecture à 3 niveaux**
  - **Objet :** capteurs / actionneurs (Edge)
  - **Réseau :** passerelle, LoRaWAN, Wi-Fi, MQTT
  - **Service :** cloud, traitement & tableau de bord
- **Enjeux d'ingénierie**
  - Autonomie énergétique, sécurité et fiabilité



L'IoT, c'est le même système. . . mais relié à Internet



Exemple : l'agriculture connectée

Un capteur mesure l'humidité du sol → la donnée part sur Internet → l'agriculteur la consulte sur son téléphone et peut déclencher l'arrosage à distance.

Étape 6

Et concrètement, comment s'y mettre ? **Arduino**

Un outil accessible pour passer de la théorie à la pratique.

Place de l'Arduino

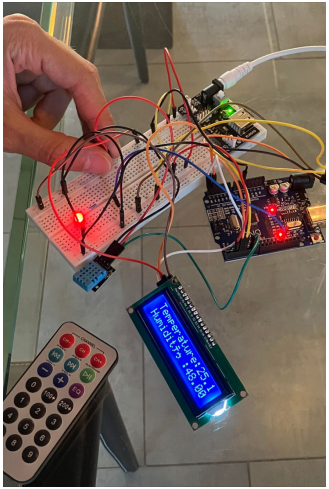


Figure 2: Matériel Arduino

- **Une carte électronique prête à l'emploi**
  - Contient un microcontrôleur, avec des broches pour brancher capteurs et actionneurs
- **Prototypage rapide (Open Hardware / Software)**
  - Masque la complexité bas niveau des registres matériels
  - Écosystème riche en bibliothèques et composants
- **Passerelle physique $\leftrightarrow$ logiciel**
  - Interfaçage direct avec capteurs et actionneurs
  - Programmation intuitive en C/C++ via l'Arduino IDE
- **Socle du projet fil rouge**
  - Idéal pour concrétiser les concepts avant passage à l'échelle

Ce qu'il faut retenir

- **Électronique** : contrôler le courant pour traiter de l'information.
- **Système électronique** : entrée (capteur) $\rightarrow$ traitement $\rightarrow$ sortie (actionneur).
- **Microcontrôleur** : le cerveau, un mini-ordinateur sur une puce.
- **Système embarqué** : un objet qui intègre ce cerveau pour une tâche précise.
- **Interfaçage** : le traducteur entre le monde physique et le programme.
- **IoT** : ces objets, connectés à Internet.
- **Arduino** : l'outil pour expérimenter tout cela facilement.

Place à la pratique

Voyons maintenant

comment prendre en main un kit Arduino

Partie 2 - Prise en main d'un kit Arduino

Objectif de cette partie

Découvrir concrètement le matériel Arduino, reconnaître les composants du kit et réaliser un premier montage sans rien endommager.

- La carte Arduino
- Les composants du kit
- La breadboard
- Comprendre la maille
- Bonnes pratiques et erreurs fréquentes

La carte Arduino, c'est quoi ?

- Une carte électronique **programmable**
- Un **microcontrôleur** (ATmega328P pour l'Uno)
- Des broches d'**entrées / sorties**
- Une connexion **USB** (alimentation + programmation)
- Un bouton **Reset**

Analogie

La carte Arduino est un **petit ordinateur** : le microcontrôleur est le cerveau, les broches sont les bras et les sens.

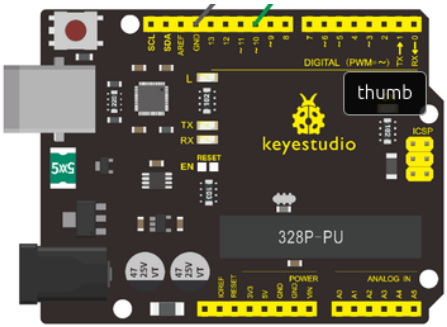
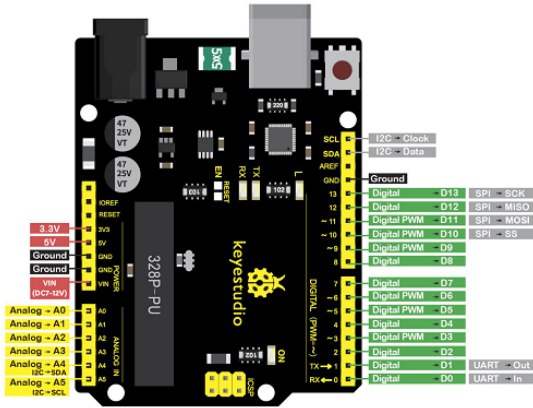


Figure 3: Carte Arduino Uno (Keyestudio)

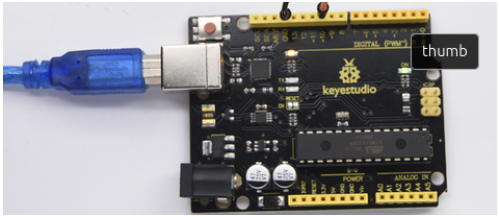
Les blocs principaux

- **Microcontrôleur** : le cerveau (ATmega328P)
- **Broches digitales** (0–13) : tout ou rien (HIGH/LOW)
- **Broches analogiques** (A0–A5) : tensions variables
- **Alimentation** : 5 V, 3.3 V, GND
- **Port USB** : alimentation + communication série



Alimentation et connexion

- Alimentation par **USB** (5 V) ou par **jack** (7–12 V)
- La carte **régule** en 5 V et 3.3 V
- L'USB sert aussi à **envoyer le code**
- Toujours vérifier la polarité avant de brancher



À retenir

Ne jamais alimenter une broche 5 V avec une source externe sans régulation.

Entrées et sorties

- **Entrées** : capteurs, boutons, potentiomètres
- **Sorties** : LED, buzzers, moteurs, relais
- **Digital** : 0 ou 1 (LOW / HIGH)
- **Analogique** : valeurs de 0 à 1023
- **PWM** : tension analogique simulée (broches ~)



Les composants du kit

- LED, résistances, boutons
- Capteurs : température, flamme, humidité, ultrasons, PIR, gaz...
- Actionneurs : servomoteur, moteur pas à pas, relais, buzzer
- Modules : LCD, matrice LED, 7 segments, RFID, horloge
- Breadboard, fils Dupont, câble USB



Les composants de base

- **LED** : s'allume quand le courant passe
- **Résistance** : limite le courant (220 Ω pour LED)
- **Bouton-poussoir** : ferme le circuit
- **Potentiomètre** : résistance variable
- **Buzzer** : émet un son (actif ou passif)

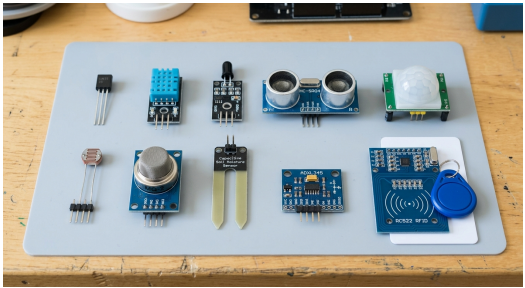


Règle d'or

Une LED ne se branche jamais seule : toujours une résistance en série.

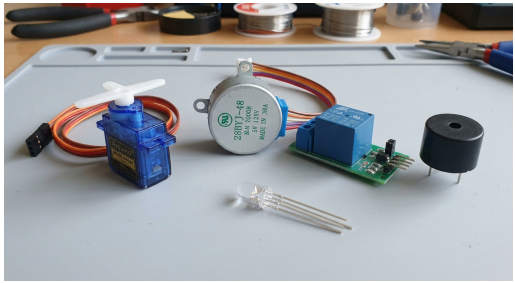
Les capteurs du kit

- **LM35** : température (linéaire, 10 mV/°C)
- **DHT11** : température + humidité
- **Capteur de flamme** : infrarouges du feu
- **HC-SR04** : ultrasons (distance)
- **PIR** : détection de mouvement
- **Photorésistance**, **capteur de gaz**, **humidité du sol**
- **ADXL345** : accélération 3 axes
- **RC522** : RFID



Les actionneurs

- **Servomoteur** : rotation angulaire précise (0–180°)
- **Moteur pas à pas** : rotation continue contrôlée
- **Module relais** : commander des appareils 230 V
- **Buzzer** : son
- **LED RGB** : couleurs



Rôle

Un actionneur transforme un signal électrique en **mouvement**, **son** ou **lumière**.

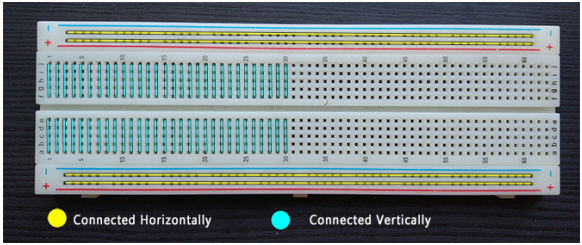
Reconnaître les composants

- **LED** : dôme, 2 pattes (longue = +)
- **Résistance** : cylindre, anneaux de couleur
- **Bouton** : boîtier carré, 4 pattes
- **Capteurs** : petites cartes à 3 broches (VCC, GND, Signal)
- **Modules** : cartes plus grandes avec connecteurs



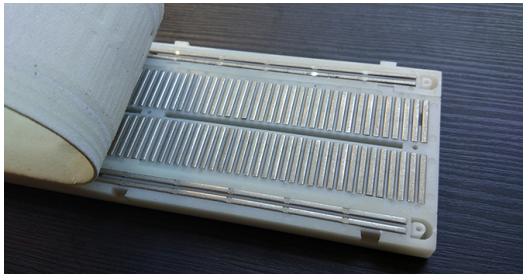
La breadboard, comment ça marche ?

- Plaque percée de trous reliés **électriquement**
- **Aucune soudure** nécessaire
- Lignes d'alimentation (rouge $+$, bleu $-$)
- Colonnes centrales pour les composants
- Connexions internes invisibles



Lignes et colonnes de la breadboard

- **Lignes d'alimentation** : toute la longueur est connectée
- **Colonnes centrales** : 5 trous alignés sont connectés
- **Séparation centrale** : isole les deux moitiés
- **Repères** : 1 à 30 (lignes), a à j (colonnes)

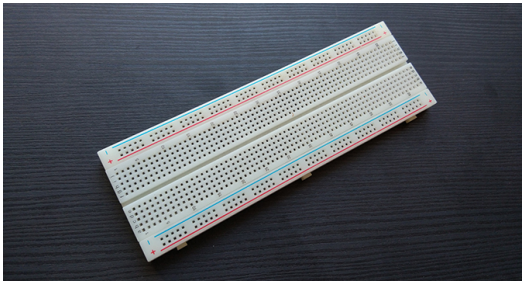


Astuce

La ligne rouge est le +, la ligne bleue est la **masse (GND)**.

Avantages de la breadboard

- **Prototypage rapide**
- **Aucune soudure**
- Composants **réutilisables**
- Modifications **faciles**
- Idéal pour **apprendre et tester**



Comprendre la maille électronique

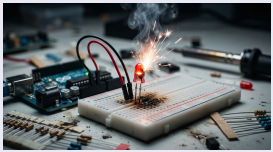
- **Qu'est-ce qu'une maille ?** Un chemin fermé que parcourt le courant électrique.
- Le courant part du **5 V** de l'Arduino, traverse la breadboard et ses composants, et **doit impérativement retourner au GND** (0 V).
- Si la maille est ouverte (un fil débranché), rien ne fonctionne.

Bonne pratique

Toujours câbler votre circuit **hors tension** (câble USB débranché), puis vérifier visuellement la boucle avant d'alimenter.

Erreur classique

Le piège de la même ligne :
Brancher les deux pattes d'une résistance ou d'une LED sur la même colonne verticale (ex: 12a et 12b) annule son effet et crée un court-circuit.

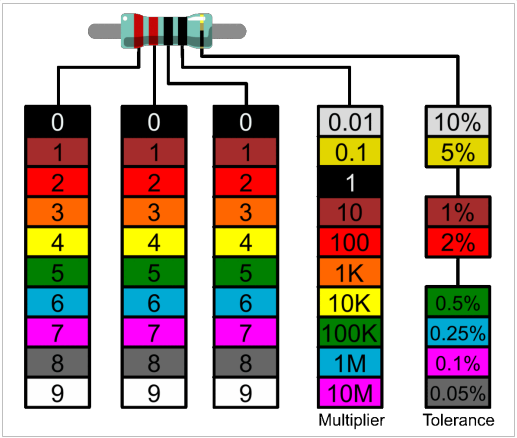


Lecture de valeur de la Résistance

Astuce

Les kits Arduino utilisent généralement des résistances à **4 bandes** :

- **Bande 1 & 2 :** Les deux premiers chiffres.
- **Bande 3 (Multiplicateur) :** Nombre de zéros à ajouter ($\times 10^n$).
- **Bande 4 (Tolérance) :** Souvent de l'or ($\pm 5\%$).



Le kit Arduino que nous utilisons

KS0077 KS0078 KS0079 Super Learning Kit for Arduino



Conclusion - Ce qu'il faut retenir

Synthèse de la prise en main

L'électronique avec Arduino n'est pas difficile, mais elle demande de la **rigueur**. Avant de coder, le matériel doit être parfaitement respecté.

- **Le Cerveau :** L'Arduino exécute vos ordres via ses broches Digitales, Analogiques et PWM.
- **La Protection :** La résistance est le garde du corps de vos composants (220,Ω minimum pour une LED).
- **Le Labyrinthe :** La breadboard distribue l'énergie verticalement pour les composants et horizontalement pour l'alimentation.

Prêts à donner vie à ce matériel ?

Place à la programmattion

Voyons maintenant
comment coder en arduino

Objectif de cette partie

Apprendre à écrire, téléverser et comprendre un programme Arduino : de l'installation de l'IDE jusqu'à la maîtrise des entrées/sorties, en passant par les fonctions de base et les bibliothèques.

- Installation et environnement
- Structure d'un programme
- Le programme Blink
- Entrées / Sorties et fonctions Arduino
- Pièges classiques (`delay` vs `millis`)
- Bibliothèques

Installer Arduino IDE

Source officielle
`arduino.cc/en/software`

Windows

- Télécharger l'installeur `.exe`
- Accepter l'installation des pilotes USB
- Redémarrer si demandé

Ubuntu

- Préférer l'archive `.tar.xz` du site officiel
- Ajouter l'utilisateur au groupe `dialout` :

```
sudo usermod -aG dialout $USER
```

- Se reconnecter pour appliquer

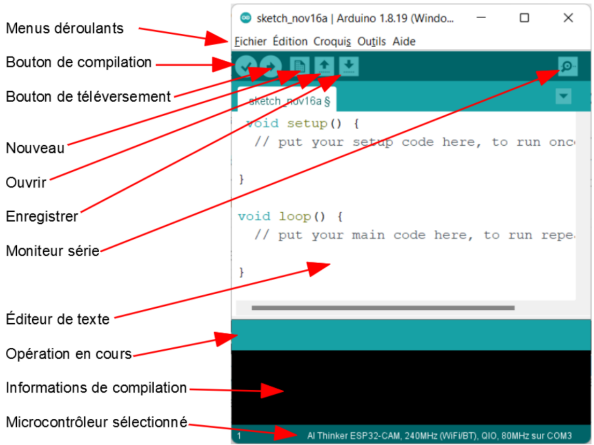
Vérification : brancher la carte, un port apparaît (`COMx` sur Windows, `/dev/ttyUSB0` sur Ubuntu).

L'environnement Arduino IDE

- Arduino utilise une version légèrement adaptée du langage C++
- Écrire, vérifier, téléverser un programme
- Trois zones : éditeur / boutons / console
- Un programme = un *sketch*
- Cycle : écrire → vérifier → téléverser

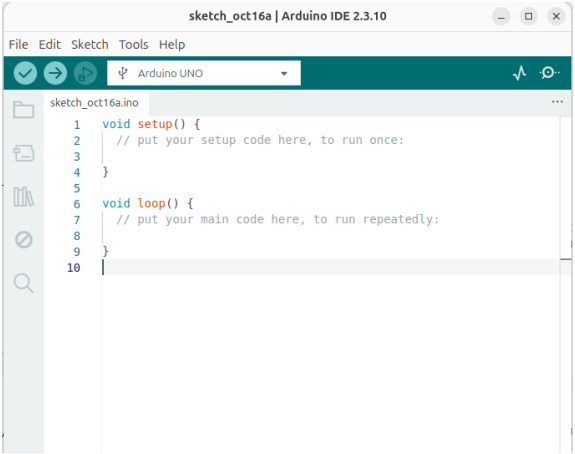


L'interface de l'IDE Arduino



Structure d'un programme

- `setup()` : exécuté **une seule fois** au démarrage
- `loop()` : exécuté **en boucle infinie** tant que la carte est alimentée
- Analogie : préparer la scène, puis rejouer la pièce sans fin
- Attention : si on met tout dans `setup()`, le programme s'exécute une seule fois et s'arrête.



Le programme Blink : faire clignoter la LED intégrée

- La broche **13** est reliée à une LED intégrée sur la carte
- Pas besoin de câbler quoi que ce soit pour tester !
- `HIGH` = allumée, `LOW` = éteinte
- `delay(1000)` = attendre 1000 ms (1 seconde)

```
1  void setup() {
2    // Initialise la broche 13 en
       sortie
3    pinMode(13, OUTPUT);
4  }
5
6  void loop() {
7    digitalWrite(13, HIGH); // Allumer
8    delay(1000);     // Attend 1 seconde
9    digitalWrite(13, LOW);  // Eteindre
10   delay(1000);     // Attend 1 seconde
11 }
```

Astuce : la LED intégrée clignote dès le téléversement !

Rappels du C/C++ déjà connus

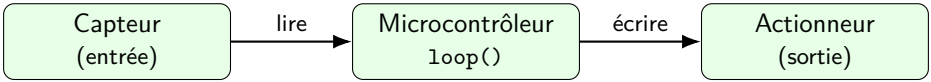
Ces notions ne sont pas redéveloppées : elles fonctionnent telles quelles en Arduino

- Variables et types : `int`, `float`, `bool`, `char`
- Conditions : `if / else`
- Boucles : `for`, `while`
- Fonctions : définition et appel

La seule différence à connaître : `int` fait 16 bits sur Arduino Uno

Un `int` va de -32768 à 32767 seulement (au lieu de milliards sur un PC 64 bits).

Entrées / Sorties : le modèle général



- Entrée : la carte reçoit une information (bouton, potentiomètre)
- Sortie : la carte commande un composant (LED, moteur, buzzer)
- Deux natures : **numérique** (tout ou rien) et **analogique** (continu)

Les fonctions Arduino que nous allons utiliser

Fonction	Rôle	Plage / Valeurs
`pinMode`	Configurer le mode d'une broche	INPUT(_PULLUP) / OUTPUT
`digitalRead`	Lire une entrée numérique	0 / 1
`digitalWrite`	Écrire une sortie numérique	HIGH / LOW
`analogRead`	Lire une entrée analogique	0 – 1023
`analogWrite`	Sortie PWM (simule l'analogique)	0 – 255
`delay`	Mettre en pause le programme	millisecondes
`millis`	Temps écoulé depuis le démarrage	millisecondes
`Serial.begin`	Ouvrir la communication avec le PC	vitesse en bauds
`Serial.print`	Afficher du texte formaté	texte

Ces fonctions seront expliquées dans les slides suivantes, avec un exemple concret à chaque fois.

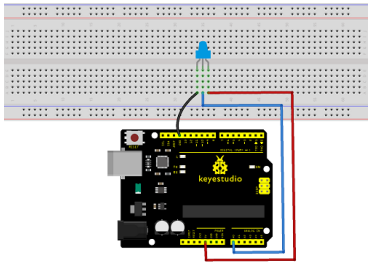
Lire une entrée numérique : le bouton

- Un bouton renvoie deux états : appuyé / relâché
- **Sans résistance**, la broche est dite **flottante** : elle capte le bruit électrique et renvoie des valeurs aléatoires.
- Spécifique Arduino : `INPUT_PULLUP` active une résistance interne pour éviter cela.
- `INPUT` : appuyé = 1, relâché = 0 (si câblé avec résistance externe vers GND)
- `INPUT_PULLUP` : appuyé = 0, relâché = 1 (état inversé)

```
1  void setup() {
2    int buttonPin = 2;
3    pinMode(buttonPin,
         INPUT_PULLUP);
4    Serial.begin(9600);    //
         ouvre la comm. a 9600
         bauds
5  }
6
7  void loop() {
8    int etat = digitalRead(
         buttonPin);
9    Serial.println(etat); //
         affiche 0 ou 1
10 }
```

Lire une entrée analogique : le potentiomètre

- Renvoie une valeur continue entre 0 et sa position max (0V - 5V)
- Lecture sur pin analogique (`A0`, `A1`...)
- `analogRead()` renvoie un entier entre **0 et 1023** (2^{10})

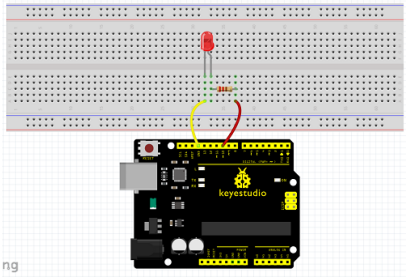



```arduino
void setup() {
  pinMode(A0, INPUT); // facultatif
  Serial.begin(9600);
}

void loop() {
  int val = analogRead(A0);
  Serial.println(val); // 0 a 1023
}
```

Commander une sortie numérique : la LED

- Deux états : allumée / éteinte
- Déclarer la pin en `OUTPUT` dans `setup()`
- Écrire avec `digitalWrite(pin, HIGH/LOW)`
- Toujours une résistance en série (220 Ω)




```arduino
void setup() {
  pinMode(8, OUTPUT);
}

void loop() {
  digitalWrite(8, HIGH);
  delay(500);
  digitalWrite(8, LOW);
  delay(500);
}
```

Commander une sortie “analogique” : le PWM

- Arduino n’a pas de vraie sortie analogique
- PWM = signal qui alterne très vite, perçu comme valeur intermédiaire
- `analogWrite(pin, valeur)`, `valeur` entre **0 et 255** (2^8)
- Correspondance : PWM 50% $\approx$ 2,5 V moyen / PWM 25% $\approx$ 1,25 V moyen
- Pins PWM : marquées ~ sur la carte (3, 5, 6, 9, 10, 11)

```
1  void setup() {
2    int ledPin = 9;
3    pinMode(ledPin, OUTPUT);
4  }
5
6  void loop() {
7    for (int v = 0; v <= 255; v +=
         5) {
8      analogWrite(ledPin, v);
9      delay(30);
10   }
11 }
```

Le piège classique : `delay()` vs `millis()`

`delay()`

- **Bloque** le programme pendant X ms
- Pendant l'attente, la carte ne fait **rien d'autre** : pas de lecture de capteurs, pas de réponse
- Simple à utiliser, mais dangereux dès qu'on a plusieurs tâches

```
void loop() { // Bloquant
  digitalWrite(13, HIGH);
  delay(1000); // bloque 1 s
  digitalWrite(13, LOW);
  delay(1000);
}
```

`millis()`

- Renvoie le **temps écoulé** depuis le démarrage (ms)
- Permet de faire plusieurs choses «en même temps » sans bloquer
- Indispensable pour un vrai projet

```
unsigned long t = 0;
void loop() { // Non bloquant
  if (millis() - t >= 1000) {
    t = millis();
    digitalWrite(13, !
        digitalRead(13));
  }
  // ... autres taches possibles
       ici
```

Le besoin des bibliothèques

Imaginez : vous devez afficher un texte sur un écran LCD

- **Sans bibliothèque** : il faut gérer manuellement les broches et respecter les chronogrammes du composant.
- **Avec bibliothèque** : quelques lignes suffisent.

Sans bibliothèque (extrait simplifié)

```
// Initialisation fastidieuse
void setup() {
  pinMode(12, OUTPUT); // RS
  pinMode(11, OUTPUT); // EN
  pinMode(5, OUTPUT);  // D4
  // ... D5, D6, D7 ...

  // ... + 40 lignes de
     chronogrammes
}
```

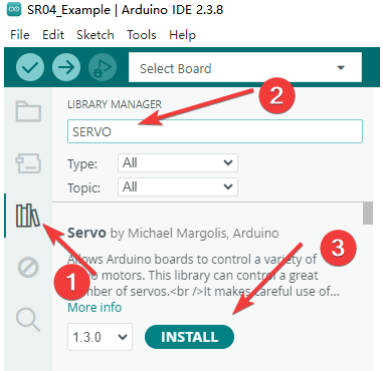
Avec la bibliothèque `LiquidCrystal`

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5,
    4, 3, 2);

void setup() {
  lcd.begin(16, 2); // 16
      colonnes, 2 lignes
  lcd.print("Bonjour !");
}
void loop() {}
```

Utiliser les bibliothèques

- Installées via le gestionnaire de bibliothèques
- Exemples : `Servo`, `Wire`, `DHT`, `WiFi`
- On les inclut avec `#include` au début



Exemple avec la bibliothèque `Servo`

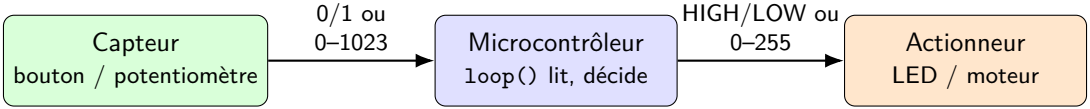
```
#include <Servo.h>
Servo monServo;

void setup() {
  monServo.attach(9);
}

void loop() {
  monServo.write(90); // angle
  delay(1000);
}
```

Ici, `Servo` gère toute la complexité des impulsions pour nous.

Synthèse : du capteur à l'actionneur



- Toutes les notions vues s'articulent dans un seul sketch
- C'est exactement le schéma repris en **Partie 4 (étude de cas)**

Partie 4 - Étude de cas

Objectif de cette partie

Mettre en pratique l'ensemble des notions vues précédemment à travers une étude de cas : partir d'un besoin réel, concevoir le schéma, câbler, programmer, tester et valider un système fonctionnel.

Nous allons faire 2 études de cas :

1. **Hello World** : faire parler la carte Arduino avec l'ordinateur
2. **Button Controlled LED**: allumer une LED à l'aide d'un bouton-poussoir

Chaque étude de cas suit le même déroulé :



Cas 1 : Hello World, présentation du systeme complet

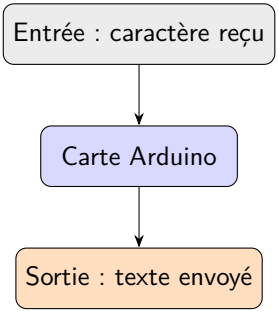
Objectif de cette étude de cas

Établir une première communication entre l'Arduino et le PC : envoyer une commande depuis le moniteur série et recevoir un message en retour.

- Le point de départ : un projet très simple, **sans aucun composant**
- Il sert de **test de communication** entre la carte Arduino et l'ordinateur
- C'est le premier essai dans le monde Arduino

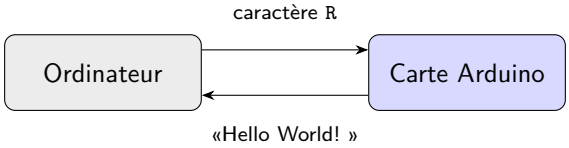
Matériel

- 1 carte Arduino (ATMEGA 328P)
- 1 câble USB



Besoin au schéma

Besoin : quand l'ordinateur envoie le caractère `R`, la carte Arduino répond «Hello World! ».



Pilote (driver) : petit programme qui permet à l'ordinateur de reconnaître la carte Arduino. Il doit être installé avant le test.

Moniteur série : fenêtre d'Arduino IDE qui permet d'envoyer des caractères à la carte et d'afficher ses réponses. *Analogie : une conversation par messages : l'ordinateur écrit `R`, la carte répond.*

Câblage

- **Aucun composant** à brancher : ni LED, ni résistance, ni breadboard
- Un seul câble : **câble USB** entre la carte Arduino et l'ordinateur
- Le câble USB assure deux choses : l'**alimentation** de la carte et la **communication** avec l'ordinateur

Avant de commencer

Installer le pilote de la carte Arduino, puis ouvrir Arduino IDE.

Code de Programmation

```
1  int val;   // variable qui va contenir le caract re re u
2
3  void setup() {
4    Serial.begin(9600);   // vitesse de communication : 9600 bauds
5  }
6
7  void loop() {
8    val = Serial.read();   // lit le caract re envoy  par l'
       ordinateur
9    if (val == 'R') {       // si ce caract re est R
10     Serial.println("Hello World!");   // la carte r pond
11   }
12 }
```

Explications du code

- `int val` : crée une **variable**, une « boîte » qui garde un nombre ou un caractère.
- `Serial.begin(9600)` : ouvre la communication avec l'ordinateur. **9600 bauds** = vitesse de transmission (la même des deux côtés).
- `Serial.read()` : lit le caractère reçu et le range dans `val`.
- `if (val == 'R')` : **condition**. Le code entre accolades ne s'exécute que si `val` vaut `R`.
- `Serial.println("...")` : envoie un texte à l'ordinateur, puis passe à la ligne.

Test et validation

1. Téléverser le programme dans la carte Arduino
2. Ouvrir le moniteur série, vitesse **9600 bauds**
3. Envoyer la lettre `R` (majuscule)

Résultat attendu : la ligne «Hello World! » s'affiche, une fois par `R` envoyé.

- Rien ne s'affiche : vérifier la vitesse du moniteur série (9600) et le bon port de la carte
- Un `r` minuscule ne déclenche rien : le programme compare avec `R` majuscule

Résultat Sur Moniteur Serie

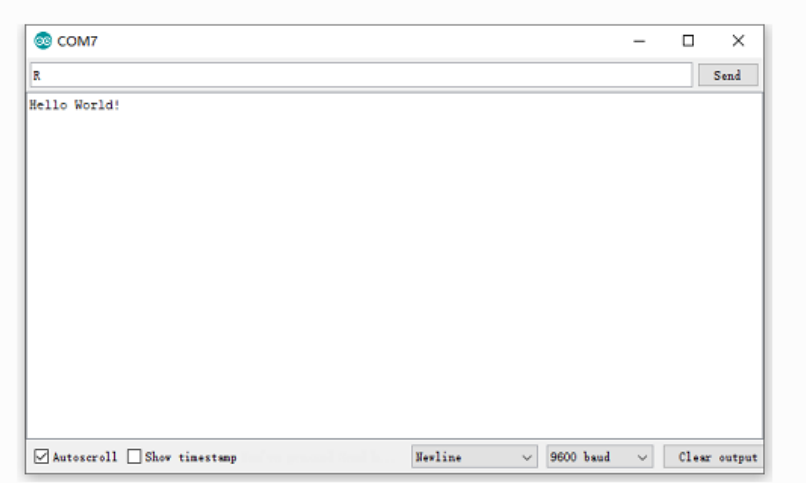


Figure 4: Resultat.

Button-Controlled LED, présentation

Objectif

Règle

- Utiliser pour la première fois une **entrée** (le bouton) **et** une **sortie** (la LED) dans le même montage
- Quand on appuie sur le bouton, la LED s'allume
- Quand on le relâche, la LED s'éteint

Jusqu'ici, nous n'avions utilisé que la fonction **sortie** (OUTPUT). Ici, la carte Arduino **lit** aussi un composant (fonction INPUT).

Matériel

- 1 carte Arduino (V4.0 ou MEGA 2560)
- 1 bouton-poussoir
- 1 LED rouge (5 mm)
- 1 résistance 220 Ω
- 1 résistance 10 kΩ
- 1 breadboard
- 6 fils de connexion
- 1 câble USB

Quelques notions importante sur le bouton-poussoir

- Le bouton est un composant **numérique** : il n'a que deux états, appuyé ou relâché
- Il fonctionne comme un interrupteur ordinaire, avec un contact **normalement ouvert**

Bouton appuyé

Le contact se ferme : le circuit est **fermé**, le courant passe.

Bouton relâché

Le contact revient à sa position de départ : le circuit est **ouvert**, le courant ne passe plus.

Analogie : le bouton d'une sonnette : tant qu'on appuie, ça sonne ; dès qu'on lâche, ça s'arrête.

Du besoin au schéma

Besoin : la LED doit suivre l'état du bouton.



Résistance 220 Ω : limite le courant dans la LED pour ne pas la griller.

Résistance 10 kΩ : relie la broche du bouton à GND. Sans elle, la broche est « dans le vide » quand le bouton est relâché et peut lire n'importe quoi. *Analogie : la résistance 10 kΩ garde la broche à 0 tant qu'on n'appuie pas, comme un ressort qui ramène un levier en position de repos.*

Schéma du montage

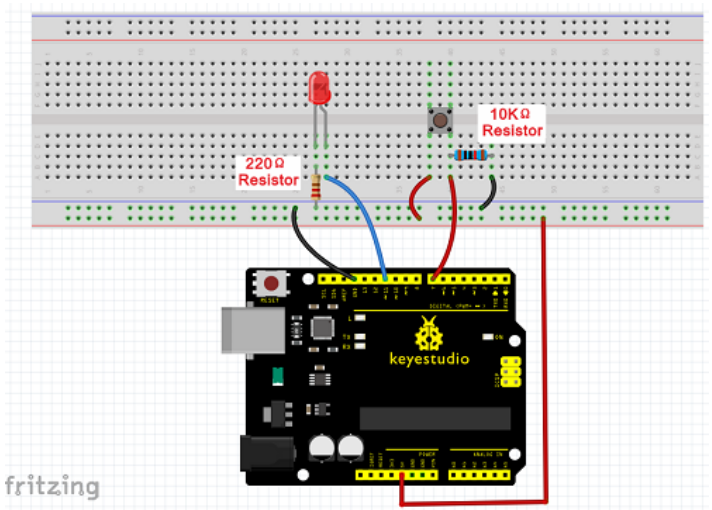


Figure 5: Schema de Montage

Câblage

Côté LED (sortie)

1. Broche 11 $\rightarrow$ résistance 220 Ω
2. Résistance $\rightarrow$ patte longue de la LED (anode)
3. Patte courte de la LED (cathode) $\rightarrow$ GND

Côté bouton (entrée)

1. 5 V $\rightarrow$ une patte du bouton
2. Autre patte du bouton $\rightarrow$ broche 7
3. Même point $\rightarrow$ résistance 10 kΩ $\rightarrow$ GND

Programmation

```
1  int brocheLed = 11;          // la LED est sur la broche 11
2  int brocheBouton = 7;        // le bouton est sur la broche 7
3  int val;                     // contient l'etat du bouton
4
5  void setup() {
6    pinMode(brocheLed, OUTPUT);      // sortie : la carte commande la
       LED
7    pinMode(brocheBouton, INPUT);    // entree : la carte lit le bouton
8  }
9
10 void loop() {
11   val = digitalRead(brocheBouton);     // lit l'etat du bouton
12   if (val == HIGH) {                   // bouton appuye
13     digitalWrite(brocheLed, HIGH);     // LED allumee
14   } else {                             // bouton relache
15     digitalWrite(brocheLed, LOW);      // LED eteinte
16   }
```

Définitions du code

- `pinMode(brocheBouton, INPUT)` : la broche sert à **lire** (entrée). C'est la nouveauté de ce TP.
- `digitalRead(broche)` : lit l'état de la broche. Résultat : `HIGH` (bouton appuyé) ou `LOW` (bouton relâché).
- `if (val == HIGH) { ...} else { ...}` : **condition**. Si le bouton est appuyé, on fait la première partie, sinon on fait la seconde.
- `digitalWrite(broche, HIGH/LOW)` : allume ou éteint la LED.

Différence avec la Partie 3

Ici on utilise `INPUT` avec une résistance 10 kΩ **extérieure**, à la place de `INPUT_PULLUP` (résistance intégrée à la carte).

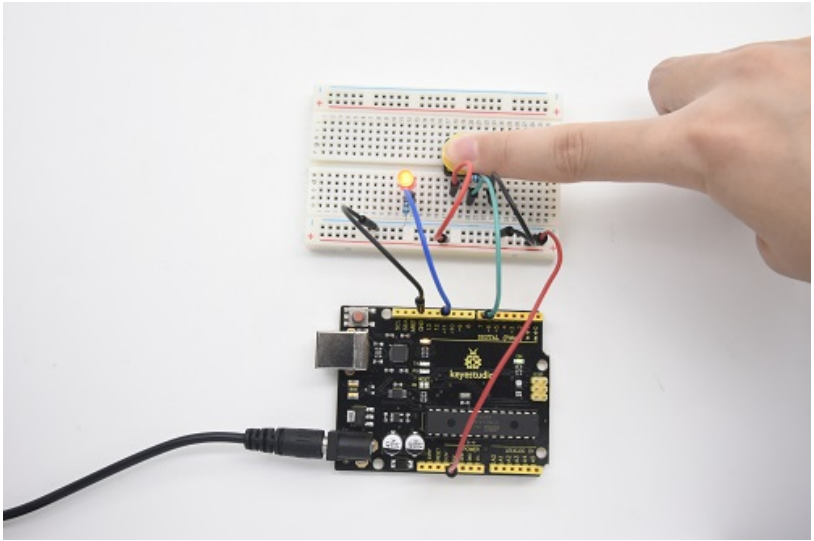
Test et validation

1. Téléverser le programme dans la carte Arduino
2. Appuyer sur le bouton, puis le relâcher

Résultat attendu : la LED est allumée tant que le bouton est appuyé, éteinte dès qu'on le relâche.

- **Validation** : remplacer `HIGH` par `LOW` dans la ligne du `if`, la LED fonctionne alors à l'inverse
- La LED ne s'allume jamais : vérifier le sens de la LED, la résistance 220 Ω et la broche 11
- La LED s'allume ou clignote toute seule : vérifier la résistance 10 kΩ vers GND

Cas 4 : résultat obtenu



Questions ?

Merci !